
Scalable Neighborhood-Based Multi-Agent Actor-Critic

Author

rajensen@student.ethz.ch
tgoppelsroed@student.ethz.ch

Abstract

We propose a novel extension to existing multi-agent actor-critic methods that addresses the scalability limitations of centralized critic approaches. Centralized critics, which leverage the observations and actions of all agents, have demonstrated significant performance gains in various multi-agent settings. However, their computational complexity grows linearly with the number of agents, hindering their application in settings where a larger number of agents are present. Our approach mitigates this problem by selectively providing the critic with observations and actions from the K closest agents based on a chosen metric, such as Euclidean distance. This modification ensures a constant input size for the Q-network, regardless of the total number of agents. We empirically validate the effectiveness of our method, achieving competitive performance with traditional centralized critics in a variety of cooperative and adversarial environments. This demonstrates the potential of our approach to enable scalable and efficient learning in complex multi-agent systems.

1 Introduction

Multi-Agent Reinforcement Learning (MARL) has recently shown immense potential for tackling complex problems involving multiple interacting agents, impacting fields such as robotics, sociology and traffic[13], among others. Recent practical results such as the AlphaStar paper[16] have shown MARL to be capable of dealing with complex multi-agent environments. However, scaling these algorithms to large numbers of agents remains a significant hurdle. Centralized critic architectures, a cornerstone of many successful MARL approaches, suffer from a significant drawback: their computational complexity grows linearly with the number of agents, quickly becoming prohibitive for large-scale scenarios. This limitation restricts the applicability of these methods to environments with relatively few agents. To address this challenge, our paper proposes two novel extensions to existing multi-agent actor-critic methods, specifically focusing on overcoming the scalability limitations of centralized critics.

MADDPG-K

A recent approach to MARL [6] introduced a centralized critic where each agent's critic receives an input containing the observations and actions for all of the agents in the environment instead of just their own observation and action. The authors of this paper refer to their algorithm as Multi-Agent Deep Deterministic Policy Gradient (MADDPG). In future work, the authors address the challenge of the centralized Q-function's input space growing linearly with the number of agents in the environment. One solution is to consider only agents in a certain region around a given agent when determining the input to the Q-function for that agent.

In this paper, we will explore the effect of defining this region using a simple Euclidean metric, where we measure the Euclidean distance between all agents in the environment and choose only the K

closest agents to an agent as input to its centralized critic. However, this can easily be extended to different metrics or one could make use of an embedding function, with agent information as input, before the metric is applied. We test our implementation on a majority of the environments used in the MADDPG paper, which allows us to directly compare the two algorithms.

MADDPG with Shared Critic

In many MARL environments, there are groups of agents working towards a common goal. The hypothesis here was that in many such environments, groups of agents could potentially benefit from sharing a critic among the agents in the group which would establish a common notion of value in the state-action space. We used a P sized moving window that iterates through P sized subsets of a given group, where each agent has the same probability of being used to update the critic of that group. Unfortunately, this approach did not seem viable based on preliminary experimental results and was thus not pursued further.

2 Related Work

The amount of research in deep reinforcement learning increased significantly after the seminal DQN paper [7] by researchers at DeepMind, which taught agents to play Atari games using Q-learning, a popular reinforcement learning algorithm combined with a deep neural network that estimates Q-values.

Research in the years following established methods to extend DQNs to be able to handle continuous action spaces using an actor-critic approach. This led to the development of algorithms such as Deep Deterministic Policy Gradient (DDPG)[5], Twin Delayed Deep Deterministic Policy Gradient (TD3)[3], and Soft Actor Critic (SAC)[4].

A naive extension of these algorithms to MARL environments is possible in the form of independent Q-learning [14], however recent research such as the MADDPG paper [6] outlines a more informed approach, where in MADDPG the DDPG algorithm is extended by augmenting each agent's critic input with the critic input that all other agents would traditionally receive (i.e., all agents' observations and actions are fed into each agent's critic). This is done to alleviate the non-stationarity of the Markov Decision Process when multiple agents are involved. MADDPG showed improved performance on all environments tested in the paper [6] over DDPG, especially on their cooperative communication and predator-prey environments (i.e. tag). Recent advancements in the field have integrated MADDPG and SAC, leveraging the attention mechanism to facilitate the joint training of critics. This innovative approach is exemplified by the MAAC (Multi-agent Actor-Attention-Critic) algorithm [12], which samples actions from all target actors, as opposed to just the current agent. This modification has demonstrated enhanced performance across various test environments. Other non-attention based variants of this approach exist and have themselves shown to be quite capable [11, 10].

A major drawback of most algorithms that use a centralized critic approach like MADDPG is that the input to the agents' critics scales linearly with the number of agents present in the environment. This is because the critic receives the observations and actions of all agents in the environment. In our project, we aim to remedy this by restricting the critic's input for a given agent to the K agents closest to that agent, according to the Euclidean distance between them. This helps to focus on more relevant information if the correct metric is used, and also reduces complexity, allowing the algorithm to scale to environments with larger numbers of agents.

An alternative approach to this is given by [17], where the critic's scaling problem is solved by averaging over pairwise local interactions between a given agent and other agents in a neighborhood around the given agent.

Concurrently to the work on the MADDPG, a separate group developed a similar approach[2], also with a centralized critic, but using a counterfactual baseline. This baseline estimates the expected return if an agent were to take a different action while other agents maintain their current actions. By comparing an agent's actual return with this counterfactual baseline, COMA can potentially more accurately attribute credit for rewards. However high variance of the counterfactual baseline can cause the gradients for updates to become unstable[9].

Since we are working on an extension to MADDPG, we need access to the environments used in the MADDPG paper[6]. However, the original environments are no longer maintained and contain obsolete code. Instead, we access them through Petting Zoo [15], which is the multi-agent equivalent of openAI gym[1] and contains a currently maintained version of these environments.

3 Methods/Algorithms

The use of a centralized critic, which obtains actions and observations from all agents as input can improve the efficiency of a MARL algorithm by dealing with the non-stationarity of the Markov decision process when multiple agents are involved, but adds additional complexity by making the input to the critic scale linearly with the number of agents in the environment.

Our first method reduces complexity, by when updating an agent, only focusing on a K sized subset of agents that are closest to the agent in question with respect to the Euclidean distance in the environment. One could however also substitute the Euclidean metric for any other metric or even learn an embedding function and measure relatedness in embedding space using a chosen metric.

By focusing only on a subset of K agents, we reduce the linear scaling of the critic input to a constant number of inputs, where K is a hyperparameter. In addition, focusing only on the observations and actions of the closest agents can lead to faster learning in the environment by focusing the agent on more relevant information.

3.1 Algorithm

Algorithm 1 MADDPG-K

```

1: for episode = 1 to M do
2:   Initialize a random process  $\mathcal{N}$  for action exploration
3:    $x \leftarrow x_{start}$ 
4:   for t = 1 to episode-length do
5:      $a_i = \mu_{\theta_i}(o_i) + \mathcal{N}_t$  for each agent  $i$  w.r.t to curr. policy and expl. noise
6:     execute  $(a_1, \dots, a_N)$  in env. and observe  $r$  and  $\mathbf{x}'$ 
7:     Get set of K closest agents  $I_i(x)$  w.r.t metric  $\omega$  for each agent  $i$ 
8:     Get set of K closest agents  $I_i(x')$  w.r.t metric  $\omega$  for each agent  $i$ 
9:     Store  $(x, a, r, x', I(x), I(x'))$  in replay buffer  $\mathcal{D}$ 
10:     $x \leftarrow x'$ 
11:    for agent i = 1 to N do
12:      Sample batch of  $S$  samples  $(x^j, a^j, r^j, x'^j, I^j(x), I^j(x'))$  from  $\mathcal{D}$ 
13:      Set  $y^j = r^j + \gamma Q_i^{\mu'}(x_i^j, x_{I_i^j}^j, a_i^j, a_{I_i^j}^j) |_{a_k^j = \mu_k'(o_k^j)}$ 
14:      Set loss  $\mathcal{L}(\theta_i) = \frac{1}{S} \sum_j (y^j - Q_i^{\mu}(x_i^j, x_{I_i^j}^j, a_i^j, a_{I_i^j}^j))^2$ 
15:      Update critic  $Q_i^{\mu}$  by minimizing the loss  $\mathcal{L}(\theta_i)$ 
16:      Set  $\nabla_{\theta_i} \tilde{J} = \frac{1}{S} \sum_j \nabla_{\theta_i} \mu_i(o_i^j) \nabla_{a_i} Q_i^{\mu}(x_i^j, x_{I_i^j}^j, a_i^j, a_{I_i^j}^j) |_{a_i = \mu_i(o_i^j)}$ 
17:      Update actor  $\mu_i$  using the sampled policy gradient  $\nabla_{\theta_i} \tilde{J}$ 
18:    end for
19:    Update target nets  $\theta'_i \leftarrow \tau \theta_i + (1 - \tau) \theta'_i$ 
20:  end for
21: end for

```

The main difference between MADDPG and MADDPG-K lies in the additional index sets for the current and next state that are additionally stored for each transition (i.e. $I(x)$ and $I(x')$). These index sets allow us to precompute the neighbors of given agents, which can then be used when updating these agents by feeding their critics only the observations and actions of their neighbors, where the agents' observations and actions are sorted according to their proximity to the given agent in the input vector.

3.2 Complexity Analysis

For the following section, we define the following variables:

n : number of agents, h : hidden size, b : batch size,
 l : number of layers, a : agent action space, o : agent observation space,
 $k \leq n$: k nearest neighbor setting, u : update frequency
 d : dimension of the vector space in which the agents move (in our case, $d=2$)

We compare the theoretical runtime of MADDPG and MADDPG-K, looking at the parts of the algorithm that differ in computational load.

The k-Nearest Neighbors precomputation for MADDPG-K requires an additional cost of $\mathcal{O}(dn^2)$ time and $\mathcal{O}(k)$ memory per step, assuming the agents move in d-dimensional Euclidean space.

During the update step, MADDPG takes all batch observations as target actor input, so the target actor process takes $\mathcal{O}(n^2bah^{2l}o)$, while MADDPG-K does a masking process before the target actor input, so its runtime is $\mathcal{O}(nkbah^{2l}o + dn^2b)$.

As for the (target) critics, MADDPG takes a total of $\mathcal{O}(n^2b(o + a)h^{2l})$ time, while MADDPG-K takes $\mathcal{O}(nkb(o + a)h^{2l})$.

The updates are done every certain number of steps, defined as u . When combining all our calculations into one runtime calculation we obtain the following:

MADDPG: $\mathcal{O}(n^2bah^{2l}o + n^2b(o + a)h^{2l})$,

MADDPG-K: $\mathcal{O}(nkbah^{2l}o + nkb(o + a)h^{2l} + dn^2b + un^2)$

3.3 Scaling Agents

While the computational overhead of traditional multi-agent algorithms grows significantly with the number of agents, MADDPG-K offers a distinct advantage for large-scale scenarios. This advantage stems from its focus on reducing the computational burden on key components – the critic and target actor networks.

In standard methods, the processing requirements for these networks scale quadratically with the number of agents. This becomes a bottleneck for large agent populations. MADDPG-K addresses this by reducing the number of inputs these networks process. Instead of considering all agents, they focus on information from a smaller set of relevant neighbors. This translates to calling these networks nk times in MADDPG-K, compared to n^2 times in traditional approaches (where n is the number of agents and k is the number of neighbors).

Despite some overhead associated with MADDPG-K (which also scales quadratically), the constant factor involved is much smaller, leading to a negligible impact on runtime compared to the bottleneck terms. As a result, MADDPG-K exhibits a relative speedup compared to traditional methods when scaling to a large number of agents. We therefore conclude there exists a crossover point (N) beyond which training with MADDPG-K becomes faster than traditional methods for any number of agents greater than N .

3.4 Bottleneck Speedup

In traditional methods like MADDPG, the critic network processes information from all agents, leading to an input size that grows linearly with the number of agents. Training an effective critic with a massive input becomes computationally infeasible. MADDPG-K addresses this by leveraging a "K-nearest neighbors"-esque approach. The critic network only considers observations and actions from agents in its immediate neighborhood, thus significantly reducing size of the critic input. This translates to a more tractable learning problem for the critic and potentially reduces the need for scaling the network itself and directly addresses the quadratic bottleneck terms (n^2) observed in traditional methods, where n is the number of agents. In MADDPG-K, these networks are called nk times, offering a substantial reduction in computation.

4 Experiments/Results/Discussion

4.1 Environments

We compared our algorithm to MADDPG on a variety of multi-agent environments from the MPE environment set [8]. To test how MADDPG-K performs when observing only a diminishing fraction

of agents, we increased the number of agents in some of the environments that allowed for this adjustment. The main environments used in this project are illustrated and described below, and all are scalable to larger numbers of agents, which is not generally the case for all MPE environments.

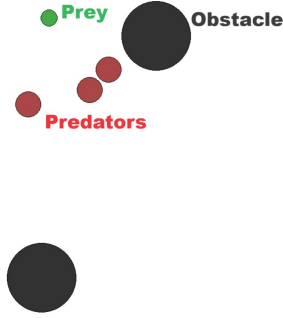


Figure 1: Simple Tag



Figure 2: Simple Spread



Figure 3: Simple Adversary

Simple Tag

In a simulated predator-prey environment, faster green "Good" agents are pursued by slower red "Adversary" agents within a square arena. Each time an Adversary tags a Good agent, the Good agent incurs a -10 penalty, while the Adversaries gain a +10 reward (globally rewarded). Obstacles in the arena hinder both types of agents. To prevent Good agents from simply fleeing the arena, an exponential penalty is imposed based on their distance outside the boundaries.

Simple-Spread

This environment features N agents and N landmarks (default $N=3$). The agents' main objective is to efficiently cover all landmarks while avoiding collisions. Agents receive a global reward based on the proximity of the closest agent to each landmark (i.e. sum of minimum distances). To encourage safer navigation, agents will incur a local penalty of -1 for each collision with another agent.

Simple Adversary

In this environment there is a single adversary (red), N good agents (green), and N landmarks (default $N=2$). All agents observe the position of the landmarks and the other agents. One landmark is designated as the 'target landmark' (colored green). Good agents are rewarded based on the proximity of the closest good agent (globally rewarded) to the target landmark, but are negatively rewarded based on how close the adversary is to the target landmark. The adversary is rewarded based on its distance to the target, but it doesn't know which landmark is the target landmark. All rewards are given by the unscaled Euclidean distance. This means that good agents must learn to "split up" and cover all landmarks to deceive the adversary.

4.2 Results:

Before extending our implementation to MADDPG-K we tested our implementations of independent DDPG and MADDPG on the simple tag environment, as well as the DDPG and MADDPG implementations from the paper (by using the code on their GitHub). We evaluated each algorithm by running it 10 times and calculating the mean learning trajectory (as depicted in 4). Finally, the sum of rewards is the sum across all four agents in the environment. Both implementations converge to higher rewards than the implementations from the original MADDPG paper. We believe that the improved performance is primarily due to the initial random action exploration (maximum entropy principle), which leads to more consistent learning. Certain other changes, such as the size of our replay memory being 10^5 rather than 10^6 , allow agents to focus on more recently stored transitions when updating, and may also have influenced the performance in this environment.

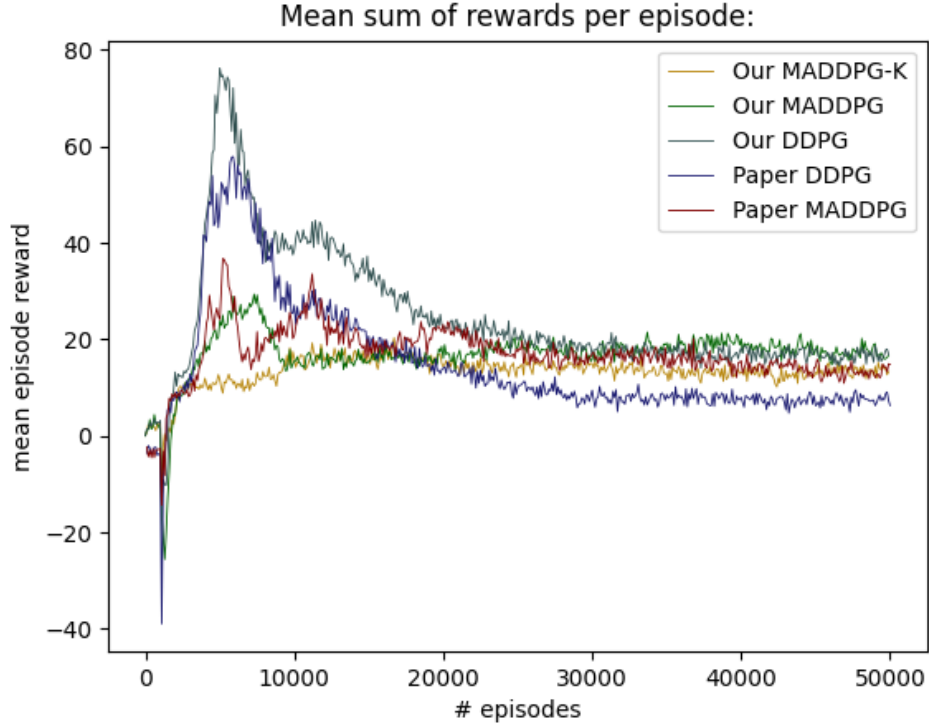


Figure 4: Simple Tag

We also ran our MADDPG-K implementation on the Simple Tag environment. This is also depicted in 4 and shows that while it does not converge to a higher reward than our MADDPG implementation, it is competitive with the MADDPG implementation from the paper. One can also see that the learning is smoother for MADDPG-K. This might be due to the ordering of adversaries or agents according to proximity. The ordering simplifies the goal of the agents, since they do not have to find out which agent is closest to them in their observation space. Instead, they can assume that the first observation in their observation space is always closest and adjust their actions accordingly. No significant difference was found between our MADDPG and MADDPG-K on Simple Tag when displaying the environment.

Because the results are less clear in adversarial environments (i.e., higher total rewards don't necessarily indicate better performing agents), we decided to see how the performance of MADDPG-K would evolve as the number of agents was increased in a cooperative environment. To do so, we chose the Simple Spread environment and scaled the number of agents from 3 to 9 while keeping $K = 2$ (i.e. we look at the 2 closest neighbors as well as the current agent) and plotted the results 5. From the results we can clearly see that MADDPG-K achieves equal or better rewards for all different sizes of N tested. For $N = 3$ the performance is near identical. This is expected, because the MADDPG-K agent is not constrained to a subset of agents. However, once K is smaller than N , MADDPG-K seems to learn faster and converge to larger rewards than MADDPG. We hypothesize that this is due to the fact that agents in MADDPG-K benefit from focusing on agents that are closer to them, since the other agents are less important to that agent's goal. This advantage diminishes as the number of agents increases. The largest advantage of MADDPG-K over MADDPG on Simple Spread was for $N = 5$ and $K = 2$, after which the advantage continues to decrease and at a certain point we strongly believe the loss of information will become too great and MADDPG will have the better performance. This indicates that the choice of the hyperparameter K has a large effect on the performance of the algorithm and the optimal choice of K is environment dependent.

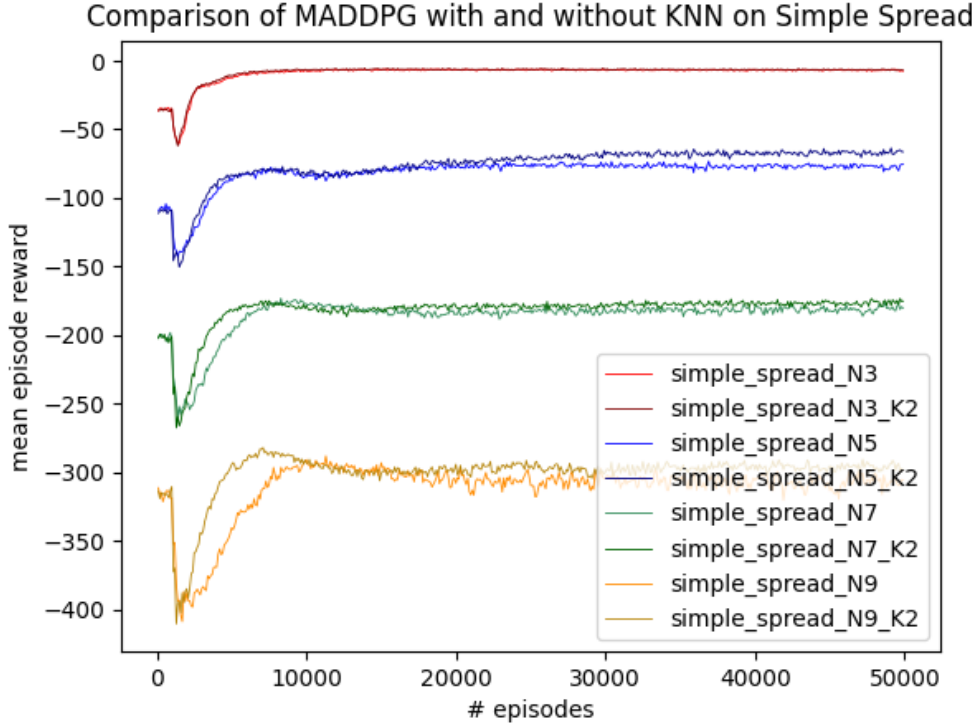


Figure 5: MADDPG-K and MADDPG on Simple Spread

For the environment Simple Adversary we conducted the same experiment 6 comparing MADDPG and MADDPG-K as we did on Simple Spread, where we fix $K = 2$ and scale N from 3 to 9. MADDPG-K demonstrates competitive performance when compared to MADDPG, even as N increases. No significant differences were observed between MADDPG and MADDPG-K. One minor difference seems to be that MADDPG-K converges faster on average than MADDPG. However more runs would have to be conducted to evaluate this statement properly. One should also note that, in the Simple Adversary Environment, only the number of Agents/Landmarks can be scaled as the number of Adversaries is fixed at 1. Since agents can receive positive reward while adversaries can only minimize their negative reward this results in higher total average rewards when the number of agents in the environment increases.

In summary MADDPG-K shows improvements over MADDPG on cooperative environments such as converging to a higher reward and learning faster. On adversarial environments MADDPG-K still seems to be quite competitive with MADDPG in terms of reward and like in the cooperative environments MADDPG-K tends to learn faster than MADDPG.

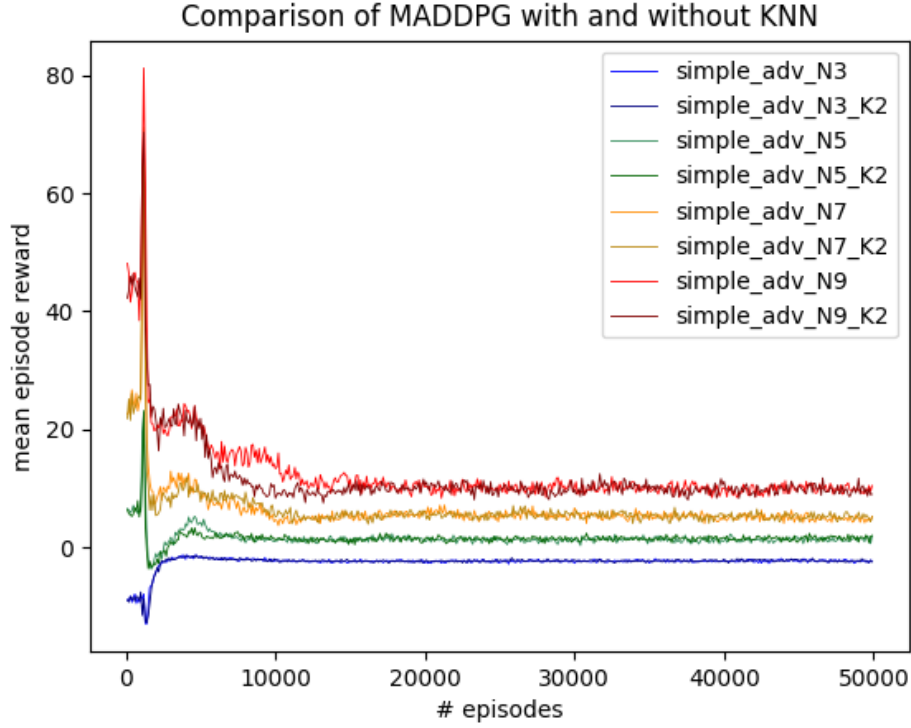


Figure 6: MADDPG and MADDPG-K on Simple Adversary

5 Conclusion/Future Work

This work presents MADDPG-K, an extension of the Multi-Agent Deep Deterministic Policy Gradient (MADDPG) algorithm [6], designed to mitigate the scalability limitations of MADDPG while also helping agents focus on more relevant information which can improve an agent's ability to learn its policy. By utilizing a neighborhood-based network, MADDPG-K reduces computational complexity, allowing for the training of more complex multi-agent systems with larger agent populations. Results show that MADDPG-K is competitive with MADDPG on all the MPE environments tested and in some specific environments MADDPG-K even seems to outperform MADDPG.

Looking ahead, MADDPG-K offers a promising solution for complex environments where traditional MADDPG might necessitate a more intricate network architecture to achieve comparable performance. We theorize, since MADDPG-K only focuses on the "more important" agents and thus seems to simplify the problem being learned, that it can achieve similar performance with a less complex network architecture than MADDPG for the same environment.

Another direction for future work would be to use different metrics, each of which would induce a different neighborhood. A particularly interesting idea would be to introduce a learned embedding function that maps agent information to an embedding space where an appropriate metric, such as the Euclidean distance metric, can be used to establish relatedness. The use of such an embedding function could potentially lead to an improved measure of relatedness between agents, leading to better results, while also allowing the algorithm to be used in environments where the position of the agent in space is not particularly relevant or does not exist.

We leave these investigations to future work.

References

- [1] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. *CoRR*, abs/1606.01540, 2016. URL <http://arxiv.org/abs/1606.01540>.
- [2] Jakob N. Foerster, Gregory Farquhar, Triantafyllos Afouras, Nantas Nardelli, and Shimon Whiteson. Counterfactual multi-agent policy gradients. *CoRR*, abs/1705.08926, 2017. URL <http://arxiv.org/abs/1705.08926>.
- [3] Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. *CoRR*, abs/1802.09477, 2018. URL <http://arxiv.org/abs/1802.09477>.
- [4] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *CoRR*, abs/1801.01290, 2018. URL <http://arxiv.org/abs/1801.01290>.
- [5] Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.
- [6] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. *CoRR*, abs/1706.02275, 2017. URL <http://arxiv.org/abs/1706.02275>.
- [7] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin A. Riedmiller. Playing atari with deep reinforcement learning. *CoRR*, abs/1312.5602, 2013. URL <http://arxiv.org/abs/1312.5602>.
- [8] Igor Mordatch and Pieter Abbeel. Emergence of grounded compositional language in multi-agent populations. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.
- [9] Georgios Papoudakis, Filippos Christianos, Lukas Schäfer, and Stefano V Albrecht. Comparative evaluation of cooperative multi-agent deep reinforcement learning algorithms. *arXiv preprint arXiv:2006.07869*, 2020.
- [10] Bei Peng, Tabish Rashid, Christian Schroeder de Witt, Pierre-Alexandre Kamienny, Philip Torr, Wendelin Böhmer, and Shimon Whiteson. Facmac: Factored multi-agent centralised policy gradients. *Advances in Neural Information Processing Systems*, 34:12208–12221, 2021.
- [11] Tabish Rashid, Mikayel Samvelyan, Christian Schroeder De Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. Monotonic value function factorisation for deep multi-agent reinforcement learning. *Journal of Machine Learning Research*, 21(178):1–51, 2020.
- [12] Fei Sha Shariq Iqbal. Actor-attention-critic for multi-agent reinforcement learning. *International Conference on Machine Learning*, 2018.
- [13] L. Codecà T. Chu, J. Wang and Z. Li. Multi-agent deep reinforcement learning for large-scale traffic signal control. *IEEE Transactions on Intelligent Transportation Systems*, 2020.
- [14] Ming Tan. Multi-agent reinforcement learning: Independent versus cooperative agents. In *International Conference on Machine Learning*, 1997. URL <https://api.semanticscholar.org/CorpusID:268857333>.
- [15] Justin K. Terry, Benjamin Black, Ananth Hari, Luis S. Santos, Clemens Dieffendahl, Niall L. Williams, Yashas Lokesh, Caroline Horsch, and Praveen Ravi. Pettingzoo: Gym for multi-agent reinforcement learning. *CoRR*, abs/2009.14471, 2020. URL <https://arxiv.org/abs/2009.14471>.
- [16] Oriol Vinyals, Igor Babuschkin, Wojciech M Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. Grandmaster level in starcraft ii using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.
- [17] Yaodong Yang, Rui Luo, Minne Li, Ming Zhou, Weinan Zhang, and Jun Wang. Mean field multi-agent reinforcement learning. In *International conference on machine learning*, pages 5571–5580. PMLR, 2018.

A Appendix

A.1 Configurations

For both our DDPG and MADDPG implementations, we use the following hyperparameters and settings:

- Adam optimizer with a learning rate of 0.01
- $\tau=0.01$ (target network update parameter)
- $\gamma = 0.95$
- replay memory size is 10^5
- update after every 100 samples collected per agent
- batch size of 1024

These hyperparameters have been chosen to match those in the MADDPG paper[6] to allow for a better comparison with our algorithm. In addition, unlike the paper, we sample random actions uniformly before the first update of the networks and have a replay memory that is smaller by one factor of 10. We do this to better explore the environment before the first update using the principle of maximum entropy and have a smaller replay memory to not keep transitions that are too old(i.e. from a time when the policy was not particularly "good" yet).

A.2 Additional Plots

Below we include further plots showing the individual performance of agents and adversaries on the base version of simple tag environment for each algorithm.

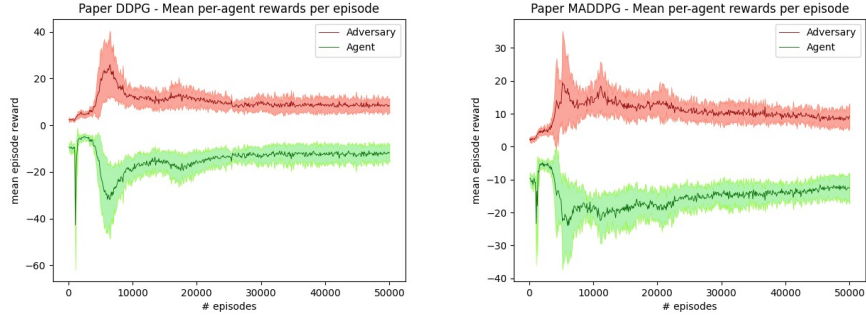


Figure 7: Algorithms from MADDPG paper GitHub

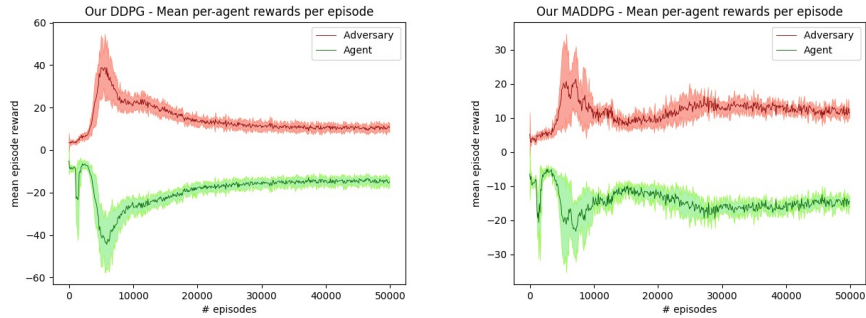


Figure 8: our versions of DDPG and MADDPG

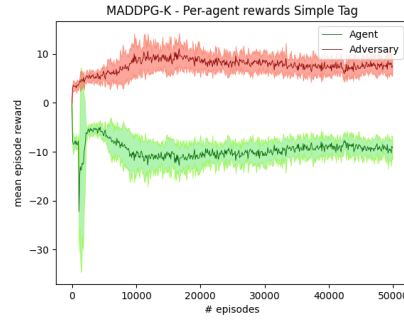


Figure 9: MADDPG-K 1 agent 3 adversaries K=3

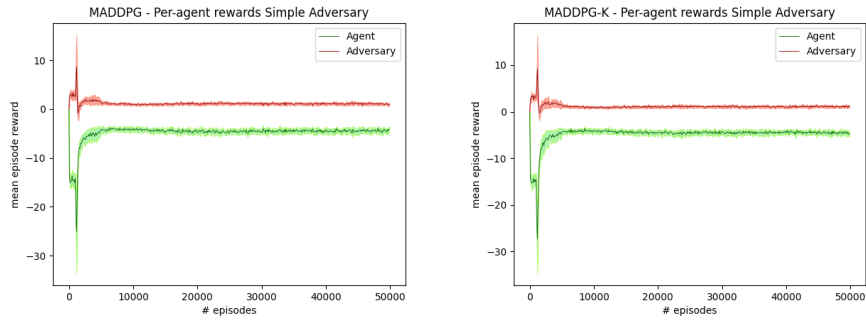


Figure 10: MADDPG and MADDPG-K Simple Adversary N=3, K=2

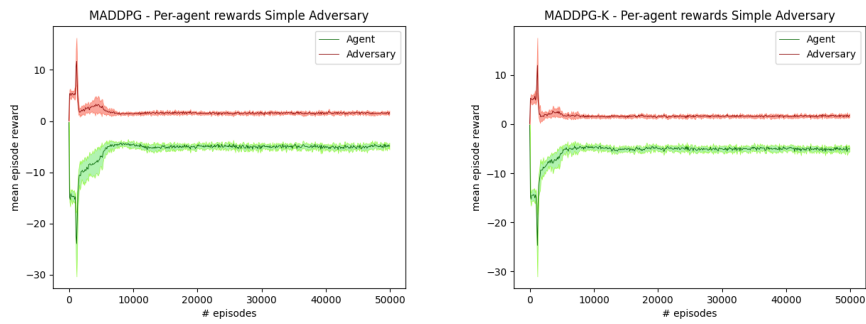


Figure 11: MADDPG and MADDPG-K Simple Adversary N=5, K=2

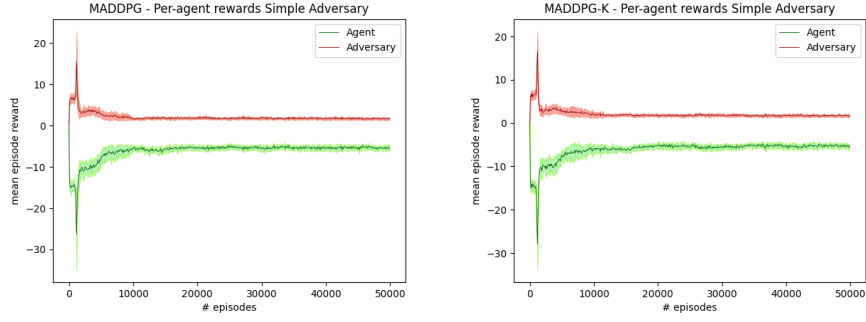


Figure 12: MADDPG and MADDPG-K Simple Adversary $N=7$, $K=2$

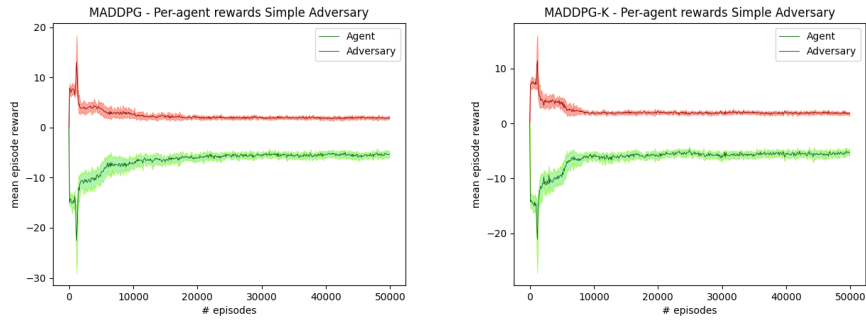


Figure 13: MADDPG and MADDPG-K Simple Adversary $N=9$, $K=2$

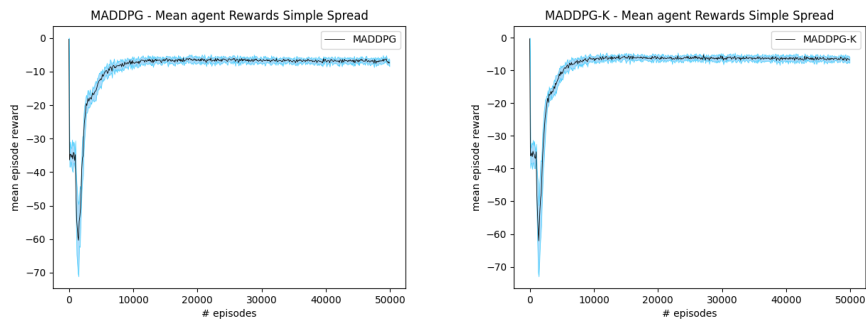


Figure 14: MADDPG and MADDPG-K Simple Spread $N=3$, $K=2$

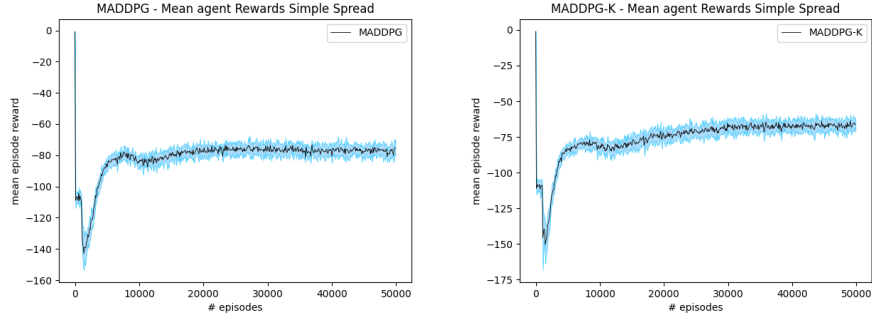


Figure 15: MADDPG and MADDPG-K Simple Spread $N=5$, $K=2$

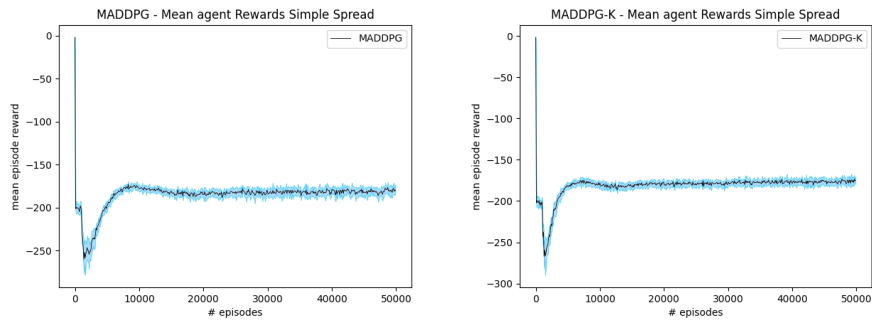


Figure 16: MADDPG and MADDPG-K Simple Spread $N=7$, $K=2$

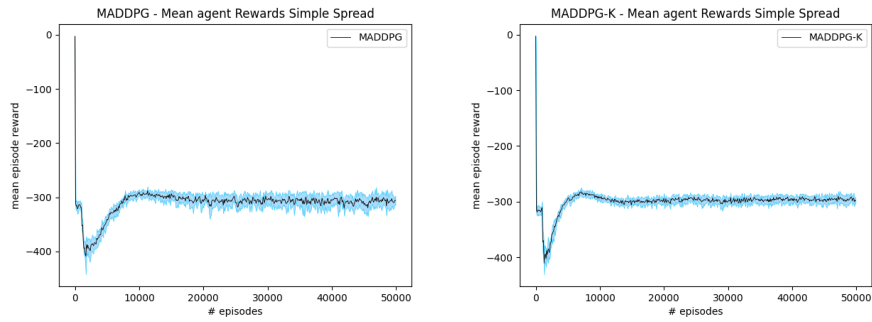


Figure 17: MADDPG and MADDPG-K Simple Spread $N=9$, $K=2$

Informal Practical Runtime Evaluation on our Hardware

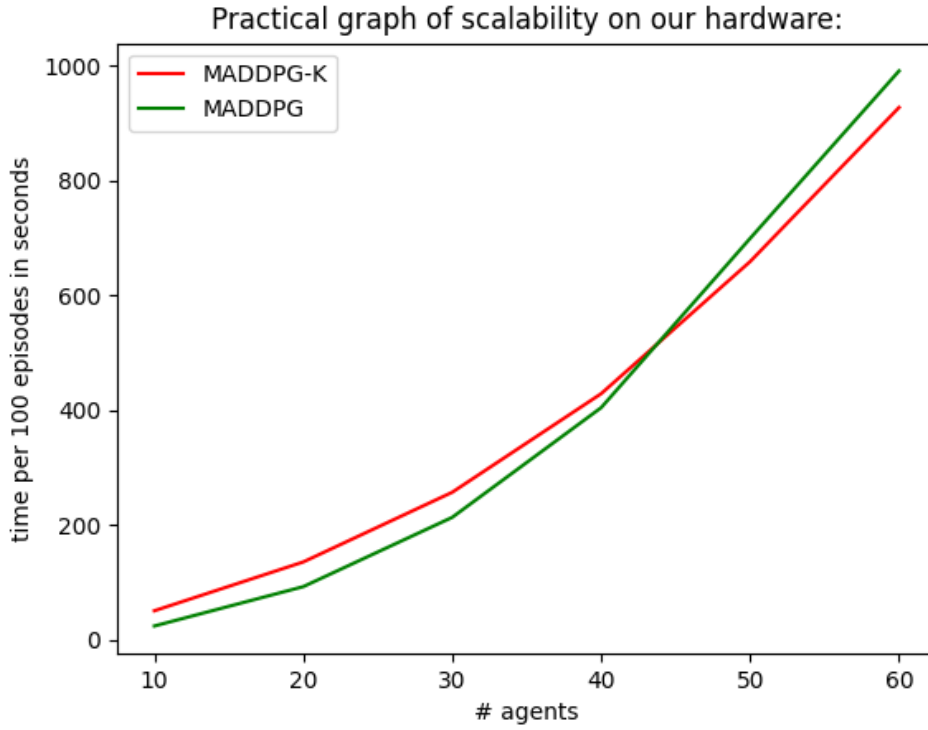


Figure 18: Scalability of MADDPG-K and MADDPG on Simple Spread environment

Note that in general because MADDPG-K by design focuses on a smaller neighborhood of agents, it will not require networks as large as MADDPG to learn an adequate policy. This additional factor is another important reason why MADDPG-K can help scale to environments with more agents. In the graph above we purposely use the same architecture for MADDPG and MADDPG-K to show that even with such a direct comparison MADDPG-K scales better.

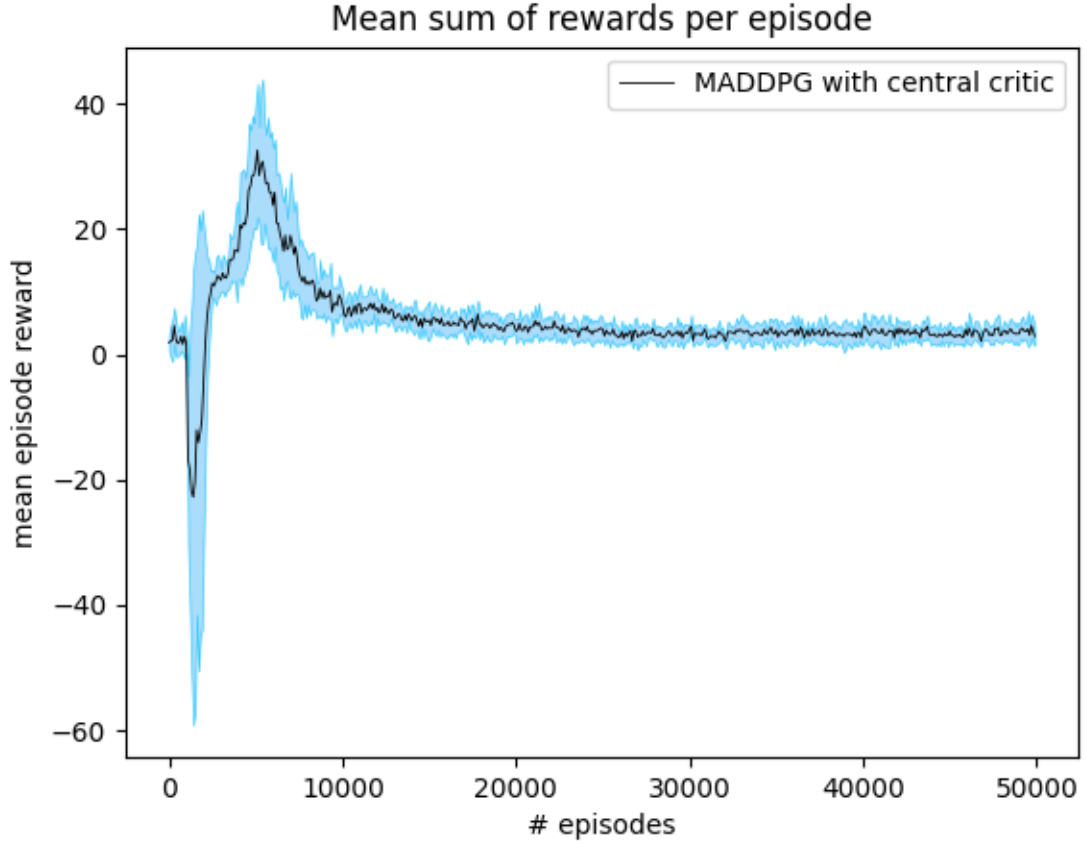


Figure 19: MADDPG with Central Critic Algorithm on Simple Tag

The convergence of the mean total reward across all agents to a value less than 10 (around 5 total average reward at convergence) clearly indicated that shared centralized critic approach as envisioned by us would not be viable. We can see that the total average reward is noticeably less than even the independent DDPG implementation achieved from the original MADDPG paper (around 10 total average reward at convergence).

A.3 Algorithms

Note that in the algorithm 20, $x = (o_1, \dots, o_N)$ is the vector of all observations and $x' = (o'_1, \dots, o'_N)$ the vector of all next observations.

Also note that K can be defined for each agent type in our code however we have simplified this to one K describing all the neighbors in our paper for the sake of simplicity.

The Original MADDPG Algorithm

Algorithm 1 MADDPG

```

1: for episode = 1 to M do
2:   Initialize a random process  $\mathcal{N}$  for action exploration
3:    $x \leftarrow x_{start}$ 
4:   for t = 1 to episode-length do
5:      $a_i = \mu_{\theta_i}(o_i) + \mathcal{N}_t$  for each agent  $i$  w.r.t to curr. policy and expl. noise
6:     execute  $(a_1, \dots, a_N)$  in env. and observe  $r$  and  $\mathbf{x}'$ 
7:     Store  $(x, a, r, x')$  in replay buffer  $\mathcal{D}$ 
8:      $x \leftarrow x'$ 
9:     for agent  $i = 1$  to N do
10:      Sample batch of  $\mathcal{S}$  samples  $(x^j, a^j, r^j, x'^j)$  from  $\mathcal{D}$ 
11:      Set  $y^j = r^j + \gamma \mathcal{Q}_i^{\mu'}(o_1^j, \dots, o_N^j, a_1^j, \dots, a_N^j)|_{a_k^j = \mu_k'(o_k^j)}$ 
12:      Set loss  $\mathcal{L}(\theta_i) = \frac{1}{\mathcal{S}} \sum_j (y^j - \mathcal{Q}_i^\mu(o_1^j, \dots, o_N^j, a_1^j, \dots, a_N^j))^2$ 
13:      Update critic  $\mathcal{Q}_i^\mu$  by minimizing the loss  $\mathcal{L}(\theta_i)$ 
14:      Set  $\nabla_{\theta_i} \tilde{J} = \frac{1}{\mathcal{S}} \sum_j \nabla_{\theta_i} \mu_i(o_i^j) \nabla_{a_i} \mathcal{Q}_i^\mu(o_1^j, \dots, o_N^j, a_1^j, \dots, a_N^j)|_{a_i = \mu_i(o_i^j)}$ 
15:      Update actor  $\mu_i$  using the sampled policy gradient  $\nabla_{\theta_i} \tilde{J}$ 
16:    end for
17:    Update target nets  $\theta'_i \leftarrow \tau \theta_i + (1 - \tau) \theta'_i$ 
18:  end for
19: end for

```

Figure 20: Original MADDPG algorithm

A.4 Methodology for Experiments

- Algorithms are tested by running them 10 times and then calculating the average reward trajectory throughout the learning process before plotting the obtained results. This methodology is followed for all the plots in this paper.
- In our paper K is defined as being the neighbors excluding the agent itself however in our code K is defined as being the neighbors including the current agent. Essentially in our paper $K = 2$ is equivalent to $K = 3$ in our code.